UNIVERSITÄT BAYREUTH

PIPELINING:

Rawen Jendoubi

Seminar: Prozessororganisation und Rechnerarchitektur

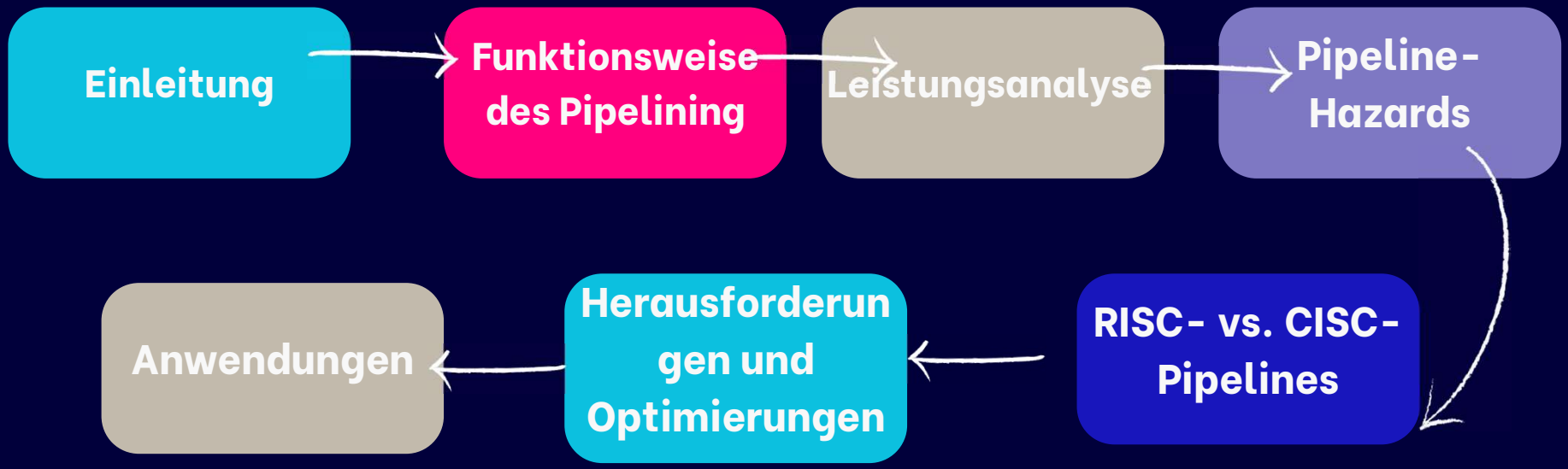
BETREUER :

Prof. Dr. Matthias Korch

Prof. Dr. Thomas Rauber

16 Januar 2025

Gliederung :


Einleitung
Funktionsweise des Pipelining
Leistungsanalyse
Pipeline-Hazards
RISC- vs. CISC-Pipelines
Herausforderungen und Optimierungen
Anwendungen

Einleitung :

Pipelining optimert die Rechenleistung

Aufgabe werden in Schritte zerlegt

mehrere Aufgaben werden paralell ausgeführt

Funktionsweise des Pipelining: :

der Prozessor lädt eine Instruktion aus dem Speicher

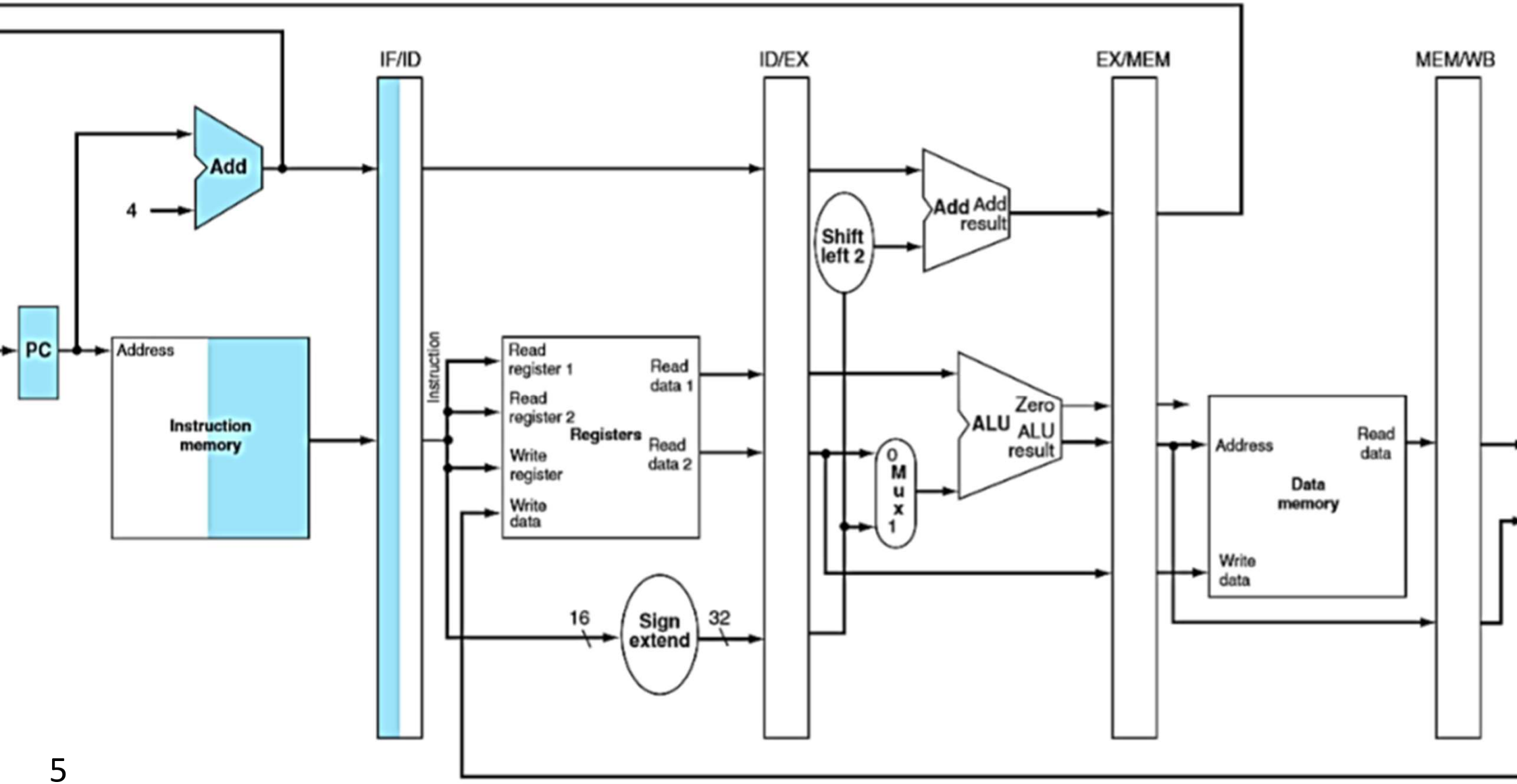
die Recheneinheit zb (ALU) führt die spezifizierte Operation wie eine Addition ,

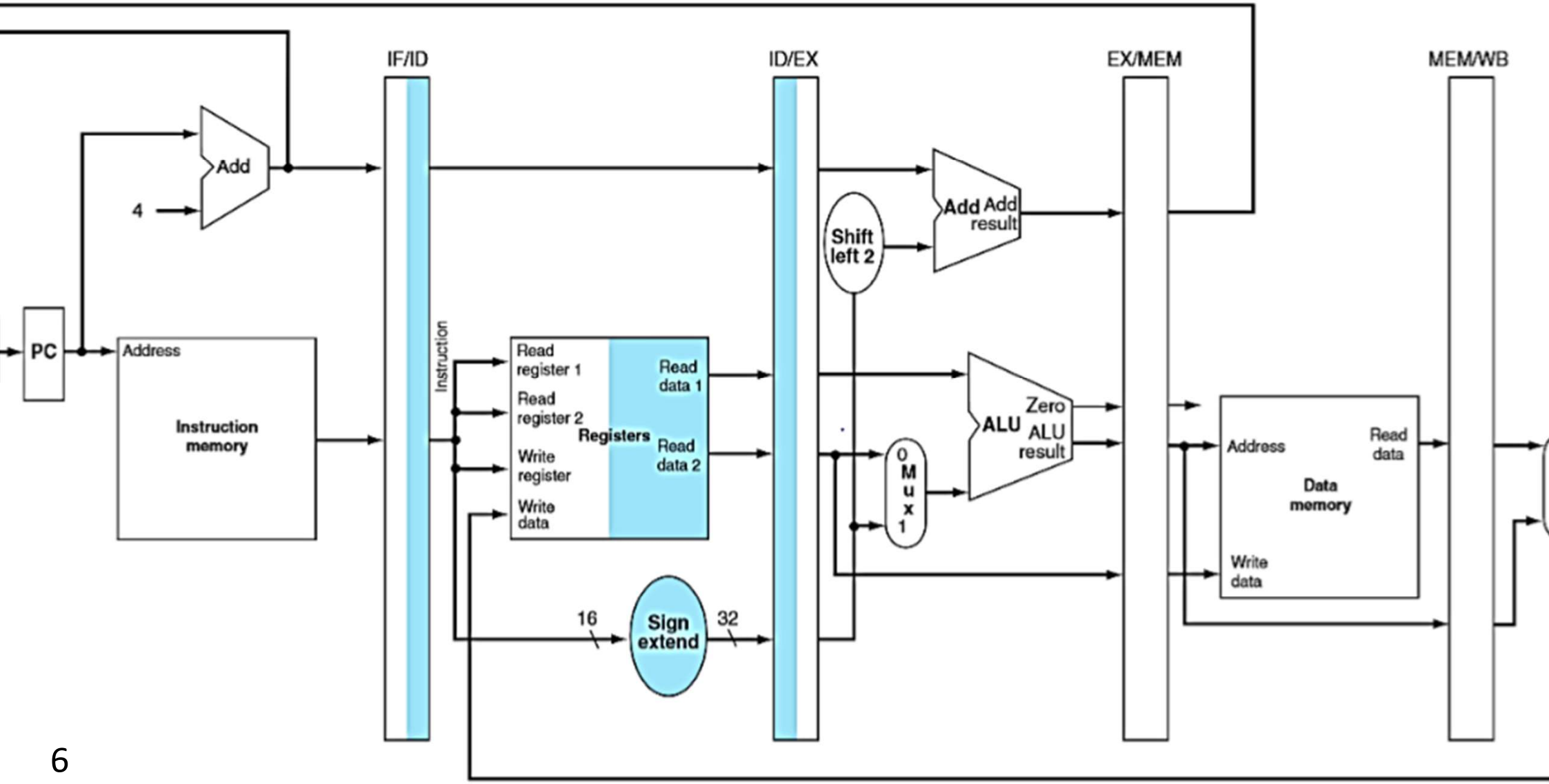
die Ergebnisse der Berechnungen werden in die entsprechenden Registern zurückgeschrieben

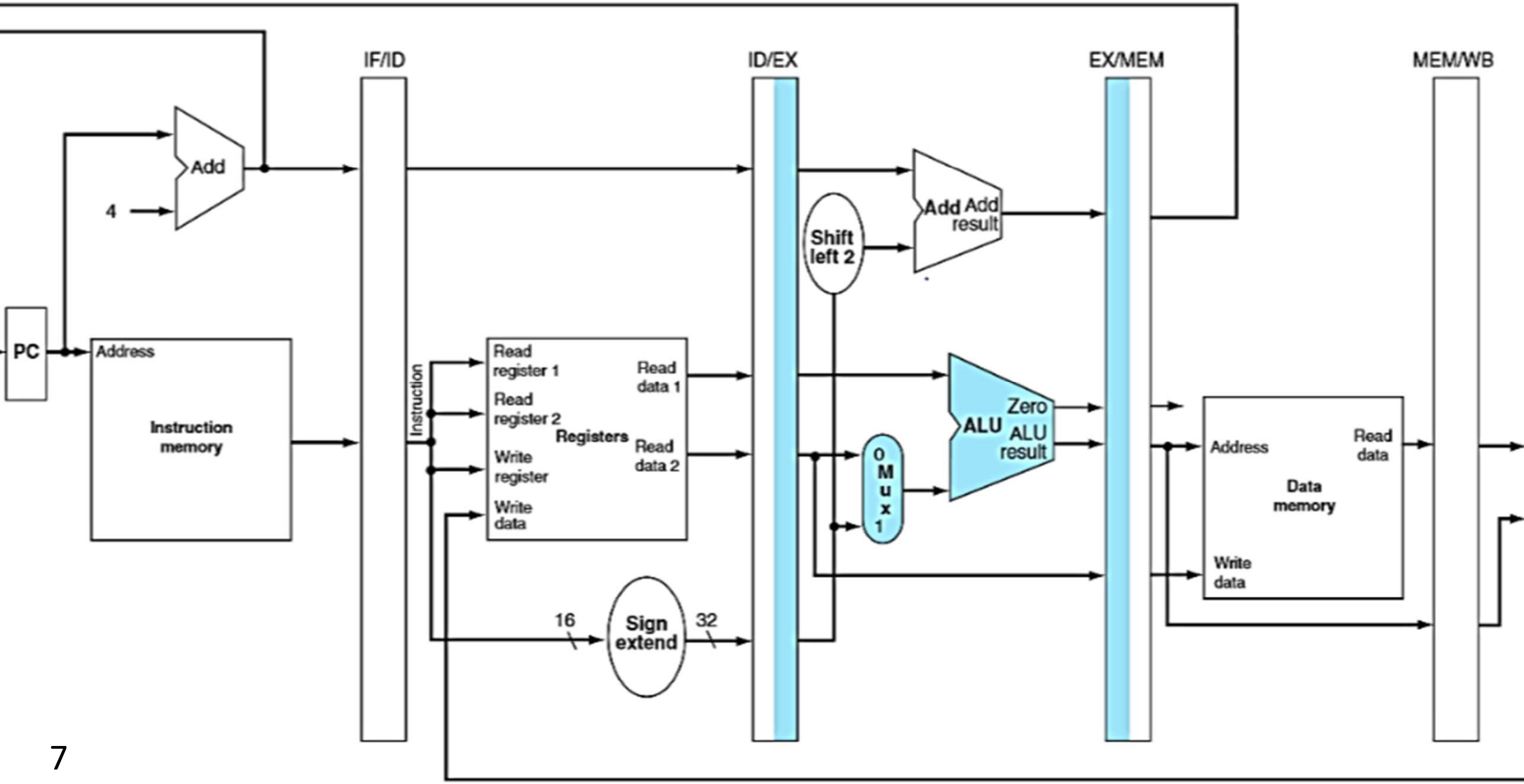
Abrufen (Fetch) → Decode → Execute → Memory Access → writeback

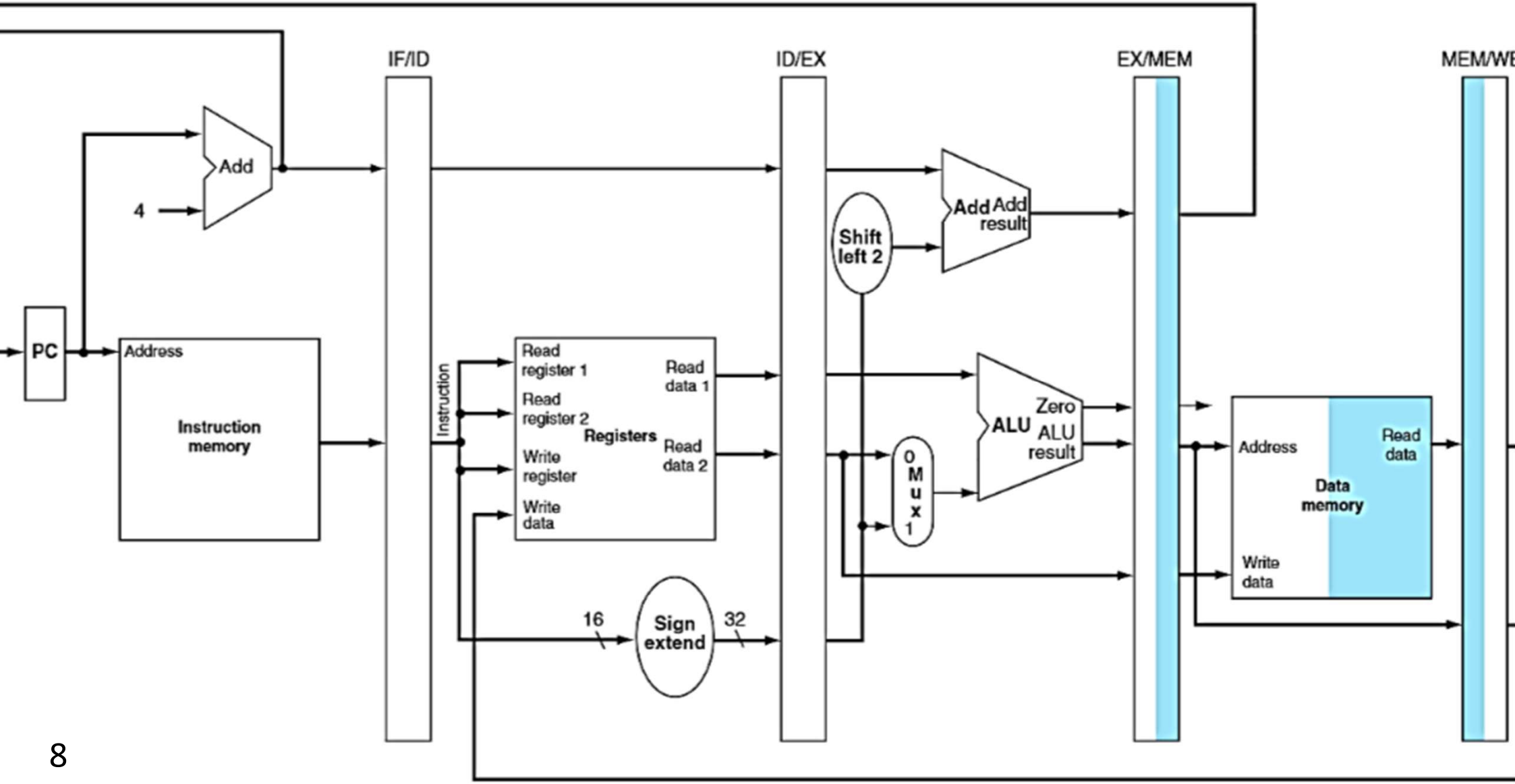
die Instruktion wird analysiert.und die benötigten registre werden gelesen

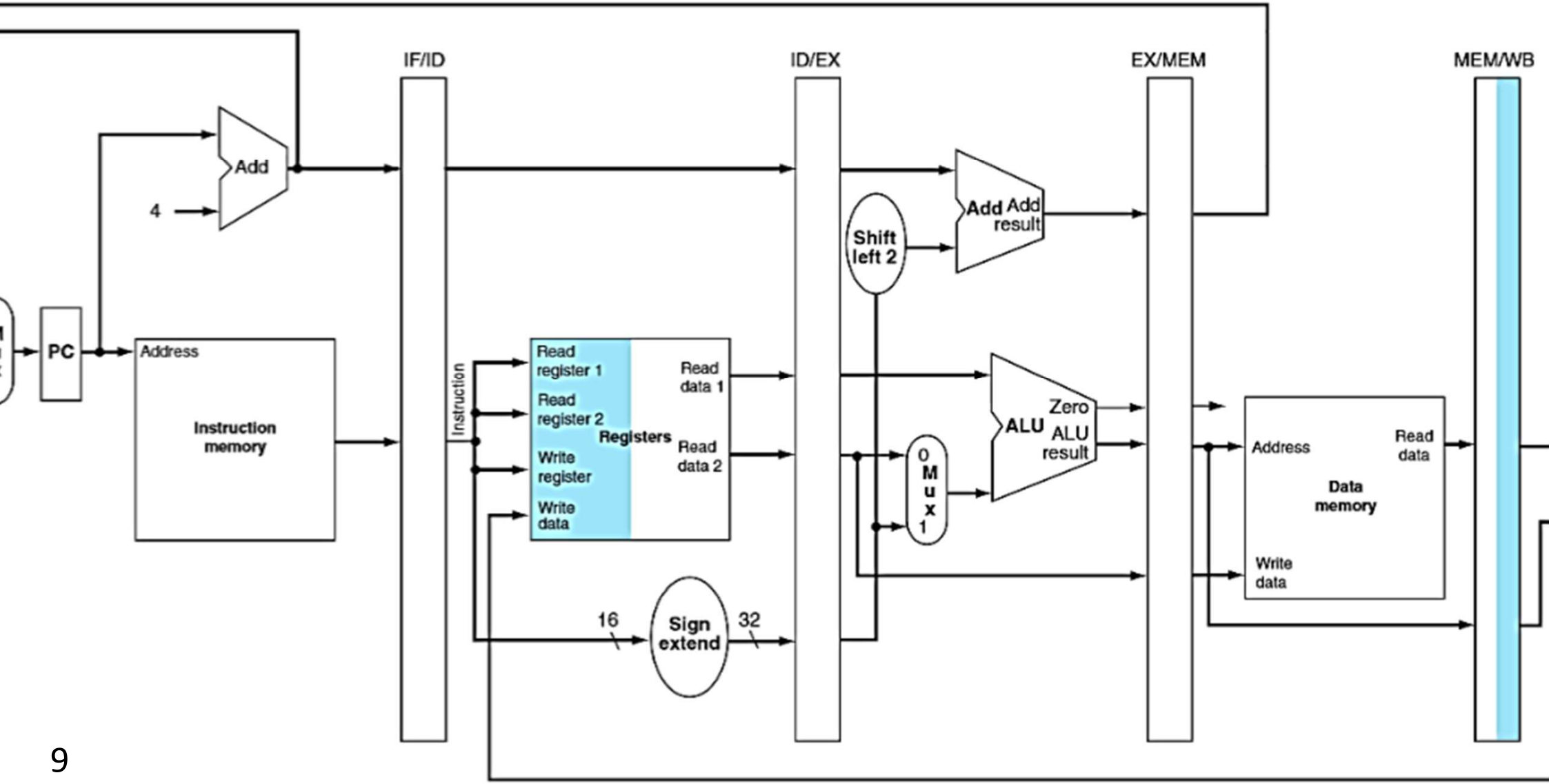
falls nötig erfolgt der Zugriff auf dem Haputspeicher , um daten zu lesen oder zu schreiben .


IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
16
Sign extend
32
Shift left 2
Add
Add result
ALU
Zero
ALU result
M u x
0
1
Address
Read data
Data memory
Write data


IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Write register
Write data
Registers
Read data 1
Read data 2
16
Sign extend
32
Shift left 2
Add
Add result
0
M
u
x
1
ALU
Zero
ALU result
Address
Read data
Data memory
Write data


IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
16
Sign extend
32
Shift left 2
Add
Add result
ALU
Zero
ALU result
M u x
0
1
Address
Read data
Data memory
Write data


IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
16
Sign extend
32
Shift left 2
Add
Add result
ALU
Zero
ALU result
M u x
0
1
Address
Data memory
Read data
Write data

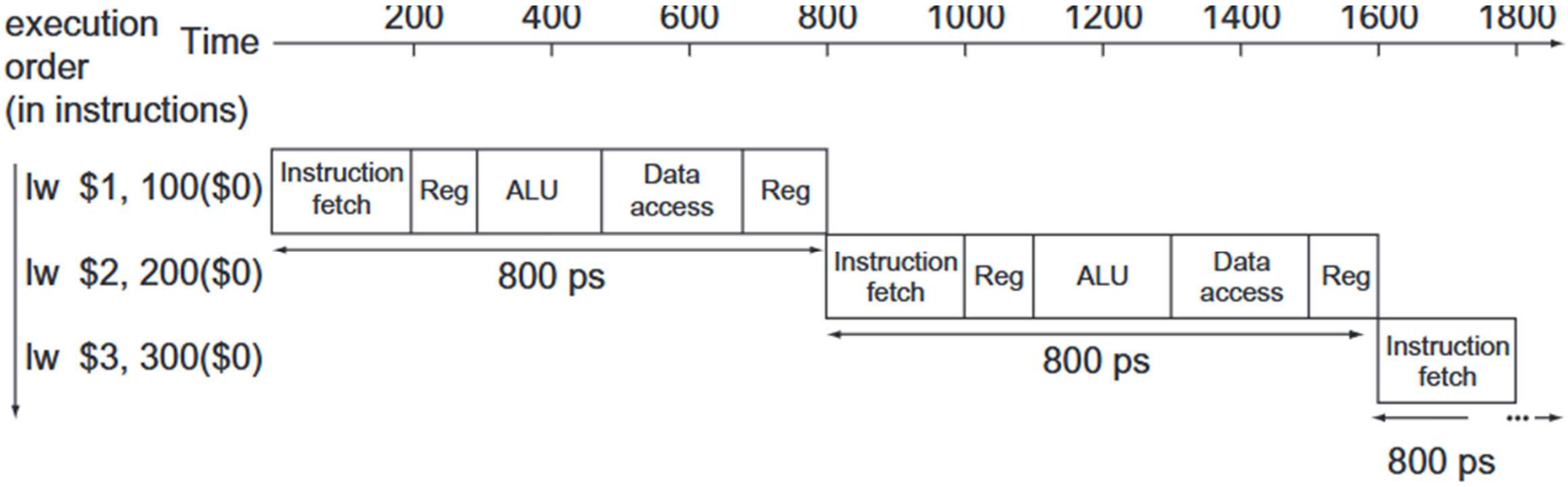

IF/ID
ID/EX
EX/MEM
MEM/WB
Add
4
PC
Address
Instruction memory
Instruction
Read register 1
Read register 2
Registers
Write register
Write data
Read data 1
Read data 2
16
Sign extend
32
Shift left 2
Add
Add result
ALU
Zero
ALU result
M u x
0
1
Address
Read data
Data memory
Write data

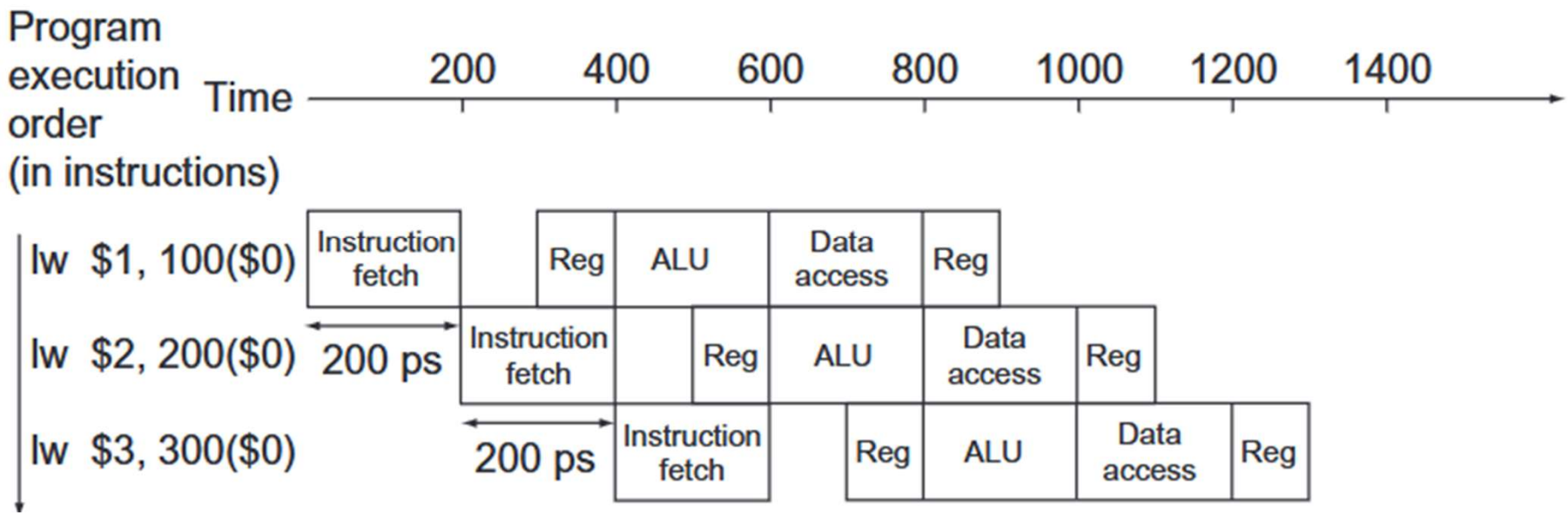
Leistungsanalyse :

-ein entscheidender Vorteil des Pipelining ist die Leistungssteigerung:

Ohne Pipelining werden Befehle nacheinander verarbeitet. Dies ist langsam.

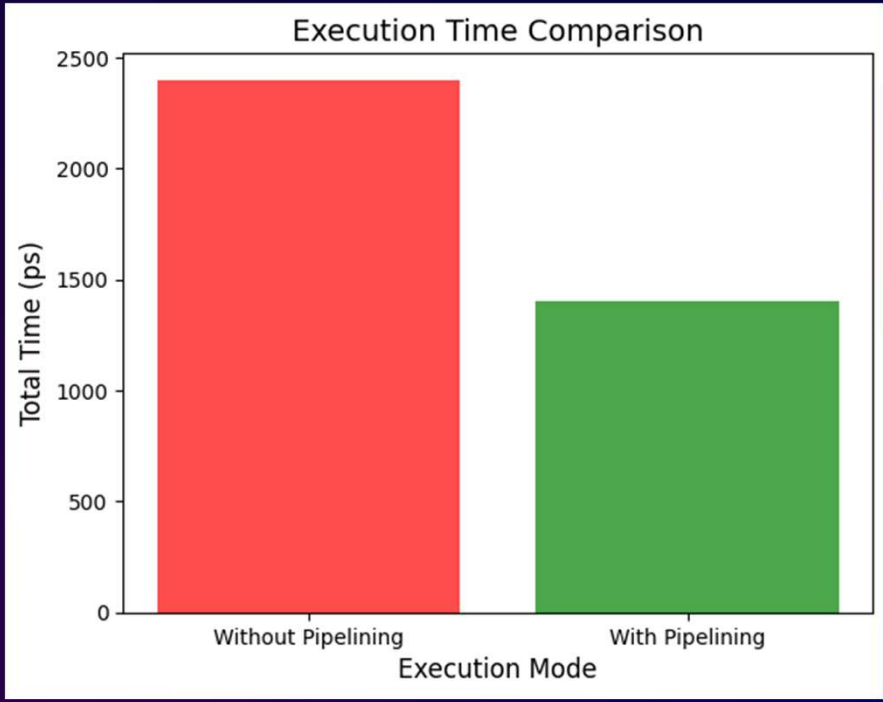
Mit Pipelining können mehrere Befehle gleichzeitig ausgeführt werden. Dies reduziert die Gesamtausführungszeit erheblich.


execution order (in instructions)
Time
200
400
600
800
1000
1200
1400
1600
1800
lw $1, 100($0)
lw $2, 200($0)
lw $3, 300($0)
Instruction fetch
Reg
ALU
Data access
Reg
800 ps
Instruction fetch
Reg
ALU
Data access
Reg
800 ps
Instruction fetch
800 ps


Program execution order (in instructions)
Time
200
400
600
800
1000
1200
1400
lw $1, 100($0)
lw $2, 200($0)
lw $3, 300($0)
Instruction fetch
Reg
ALU
Data access
Reg
200 ps
Instruction fetch
Reg
ALU
Data access
Reg
200 ps
Instruction fetch
Reg
ALU
Data access
Reg

Leistungsanalyse :

-in diesem Beispiel :
Ohne Pipelining benötigen drei Instruktionen 2400ps.
Mit Pipelining dauert es nur 1400ps. Der Geschwindigkeitsgewinn beträgt etwa 71%.



Pipeline Hazards:

Wie jede Technologie hat auch das Pipelining Herausforderungen, sogenannte Hazards:

Pipeline Hazards:

- Daten-Hazards: Tritt auf, wenn Instruktionen auf dieselben Daten zugreifen.
- Kontroll-Hazard: Entstehen bei Verzweigungen, wenn der Prozessor nicht weiß, welche Instruktion als nächstes kommt
- Strukturelle Hazards: Entstehen durch begrenzte Ressourcen.

```
.text
.globl main

main:
    # Lade die Adressen von word1 und word2
    la   $t0, word1        # Adresse von word1
    la   $t1, word2        # Adresse von word2

    # Lade die Werte von word1 und word2 in Register
    lw   $t2, 0($t0)       # Lade word1 in $t2
    lw   $t3, 0($t1)       # Lade word2 in $t3

    # XOR-basierter Swap
    xor  $t2, $t2, $t3     # Schritt 1: $t2 = word1 ^ word2
    xor  $t3, $t2, $t3     # Schritt 2: $t3 = word1
    xor  $t2, $t2, $t3     # Schritt 3: $t2 = word2

    # Speichere die neuen Werte
    sw   $t2, 0($t0)       # Speichere neuen Wert von word1
    sw   $t3, 0($t1)       # Speichere neuen Wert von word2

    # Beende das Programm
```

Data Hazard

RAW

RAW(read after write): Auch bekannt als True Dependency.Eine Instruktion liest Daten, die von einer vorherigen Instruktion noch nicht geschrieben wurden

xor $t2, $t2, $t3 # Schritt 1: $t2 = word1 ^ word2
add $t4, $t2, $t2 # Hazard: $t2 wird gelesen, bevor XOR abgeschlossen ist

-
w

beispiel von Lösung: Fügen wir eine nop-Instruktion ein, um der Pipeline Zeit zu geben, die Berechnung abzuschließen :
1 xor $t2, $t2, $t3 # Schritt 1: $t2 = word1 ^ word2
2 nop # Wartezeit, um die Berechnung abzuschließen
3 add $t4, $t2, $t2 # Jetzt ist $t2 bereit

WAW

write After Write (WAW): Auch bekannt als Output Dependency. Zwei Instruktionen versuchen, denselben Speicherort oder dasselbe Register zu schreiben.

1 xor $t2, $t2, $t3 # Schritt 1: $t2 = word1 ^ word2
2 xor $t2, $t3, $t2 # Hazard: $t2 wird direkt überschrieben
3 xor $t3, $t2, $t3 # Schritt 2: Abhängigkeit auf $t2

beispiel von Lösung: Fügen wir eine nop-Instruktion ein, um der Pipeline Zeit zu geben, die Berechnung abzuschließen :
xor $t2, $t2, $t3
nop # Ensure $t2 is ready
xor $t2, $t3, $t2
nop # Ensure $t2 is ready for $t3 update
xor $t3, $t2, $t3

RISC- vs. CISC-Pipelines:

RISC: Einfache Befehle, konstante Pipeline-Stufen
CISC: Komplexe Befehle, geeignet für komplexe Aufgaben.

Beispiel: ARM (RISC) vs. Intel Core (CISC).

RISC- vs. CISC-Pipelines:

merkmale	RISC	CISC
Befehlssatz	Einfach und klein	Komplex und groß
Befehlslänge	Fix (z. B. 32-Bit)	Variabel
Beispielbefehl	ADD R1, R2, R3	ADD EAX, [EBX]

Herausforderungen und Optimierungen :

- Herausforderungen: Speicherlatenz, Bandbreitenbeschränkungen, Stalls.
- Optimierungen:
- Caching: Schnellere Datenzugriffe.
- Breitere Datenbusse.
- Fortgeschrittene Branch Prediction.

18

Zusammenfassung:

- Pipelining zerlegt die Ausführung von Befehlen in aufeinanderfolgende Schritte, die parallel abgearbeitet werden.

- **Vorteile**: Es steigert die Effizienz und reduziert die Ausführungszeit erheblich.

- **Herausforderungen**: Hazards (Daten-, Kontroll-, strukturelle Konflikte) können auftreten, lassen sich aber durch Optimierungen minimieren.

- **Anwendungen**: Wird in RISC- und CISC-Architekturen, GPUs, und Smartphones genutzt.

- **Fazit**: Pipelining ist essenziell für die Leistung moderner Prozessoren.

Referenzen:

Krste Asanovic, David A. Patterson, et al. A view of the parallel computing
landscape. Communications of the ACM, 52(10):56–67, 2009.

1. John L. Hennessy and David A. Patterson. Computer Architecture: A Quanti-
2. tative Approach. Elsevier, 6 edition, 2019.Krste Asanovic, David A. Patterson, et al. A view of the parallel computing

3. Tariq Jamil. Risc versus cisc. Potentials, IEEE, 14:13 – 16, 09 1995.landscape. Communications of the ACM, 52(10):56–67, 2009.

4. M. Madiajagan and S. Sridhar Raj. Chapter 1 - parallel computing, graphics
5. processing unit (gpu) and new hardware for deep learning in computational in-
6. telligence research. In Arun Kumar Sangaiah, editor, Deep Learning and Paral-
7. lel Computing Environment for Bioengineering Systems, pages 1–15. Academic
8. Press, 2019

9. :https://www.cs.fsu.edu/~zwang/files/cda3101/Fall2017/Lecture8_cda3101.
10. pdf, accessed on December 6, 2024Krste Asanovic, David A. Patterson, et al. A view of the parallel computing

Vielen Dank für Ihre Aufmerksamkeit!, Gerne Fragen stellen